VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Bathala Srikar (1BM24CS068)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Bathala Srikar **(1BM24CS068)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
   int n, adj[MAX][MAX], indegree[MAX] = {0};
   int queue[MAX], front = 0, rear = 0;
   int topo[MAX], count = 0;
   printf("Enter the number of vertices: ");
   scanf("%d", &n);
   printf("Enter the adjacency matrix:\n");
   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < n; j++)
      {
         scanf("%d", &adj[i][j]);
         if(adj[i][j] == 1)
            indegree[j]++;
      }
   }
   for(int i = 0; i < n; i++)
   {
      if(indegree[i] == 0)
         queue[rear++] = i;
   }
   while(front < rear)
```

```
    {
        int u = queue[front++];
        topo[count++] = u;

        for(int v = 0; v < n; v++)
        {
            if(adj[u][v] == 1)
            {
                indegree[v]--;
                if(indegree[v] == 0)
                    queue[rear++] = v;
            }
        }
    }
    if(count != n)
    {
        printf("Topological sorting is not possible.\n");
    }
    else
    {
        printf("Topological ordering is: ");
        for(int i = 0; i < n; i++)
            printf("%d ", topo[i]);
        printf("\n");
    }
    return 0;
}
```

Output:

Case1:

```
Enter the number of vertices: 4
Enter the adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
Topological ordering is: 0 1 2 3
```

Case2:

```
Enter the number of vertices: 3
Enter the adjacency matrix:
0 1 0
0 0 1
1 0 0
Topological sorting is not possible.
```

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

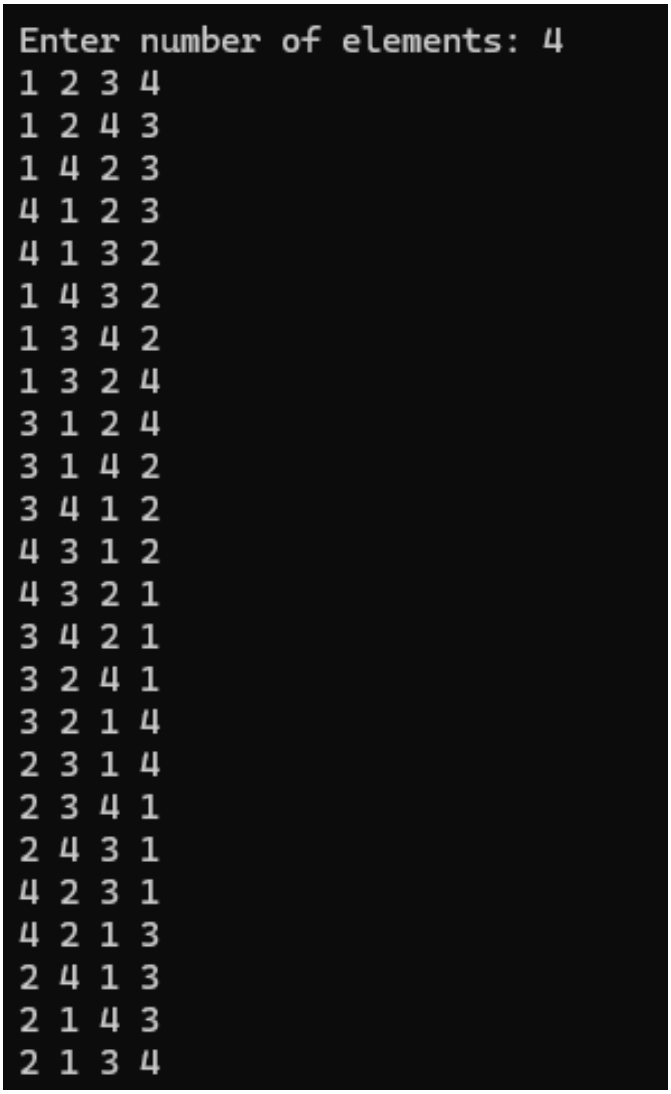
Code:

```
#include <stdio.h>
#define LEFT 0
#define RIGHT 1
void printArray(int arr[], int n) {
   for(int i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}
int getLargestMobile(int arr[], int dir[], int n) {
   int largestMobile = -1;
   int index = -1;
   for(int i = 0; i < n; i++) {
      if (dir[i] == LEFT && i != 0 && arr[i] > arr[i-1]) {
         if(arr[i] > largestMobile) {
            largestMobile = arr[i];
            index = i;
         }
      } else if (dir[i] == RIGHT && i != n-1 && arr[i] > arr[i+1]) {
         if(arr[i] > largestMobile) {
            largestMobile = arr[i];
            index = i;
         }
      }
   }
   return index;
```

```
}
void reverseDirection(int arr[], int dir[], int n, int largest) {
    for(int i = 0; i < n; i++) {
        if(arr[i] > largest) {
            dir[i] = 1 - dir[i];
        }
    }
}
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    int dir[n];
    for(int i = 0; i < n; i++) {
        arr[i] = i + 1;
        dir[i] = LEFT;
    }
    printArray(arr, n);
    while(1) {
        int largestIndex = getLargestMobile(arr, dir, n);
        if(largestIndex == -1) break;
        int swapIndex = (dir[largestIndex] == LEFT) ? largestIndex - 1 : largestIndex +
1;
        int temp = arr[largestIndex];
        arr[largestIndex] = arr[swapIndex];
        arr[swapIndex] = temp;
        int tempDir = dir[largestIndex];
        dir[largestIndex] = dir[swapIndex];
        dir[swapIndex] = tempDir;
```

```
            reverseDirection(arr, dir, n, arr[swapIndex]);
            printArray(arr, n);
        }
        return 0;
    }
```

Output:



Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void merge(int arr[], int low, int mid, int high)
{
   int i, j, k;
   int n1 = mid - low + 1;
   int n2 = high - mid;
   int L[n1], R[n2];
   for(i = 0; i < n1; i++)
      L[i] = arr[low + i];
   for(j = 0; j < n2; j++)
      R[j] = arr[mid + 1 + j];
   i = 0;
   j = 0;
   k = low;
   while(i < n1 && j < n2)
   {
      if(L[i] <= R[j])
      {
         arr[k] = L[i];
         i++;
      }
      else
      {
```

```
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while(i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
    while(j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}
void mergeSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
```

```
int main()
{
    int n, i;
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n\nTime taken to sort = %f seconds\n", time_taken);
    return 0;
}
```

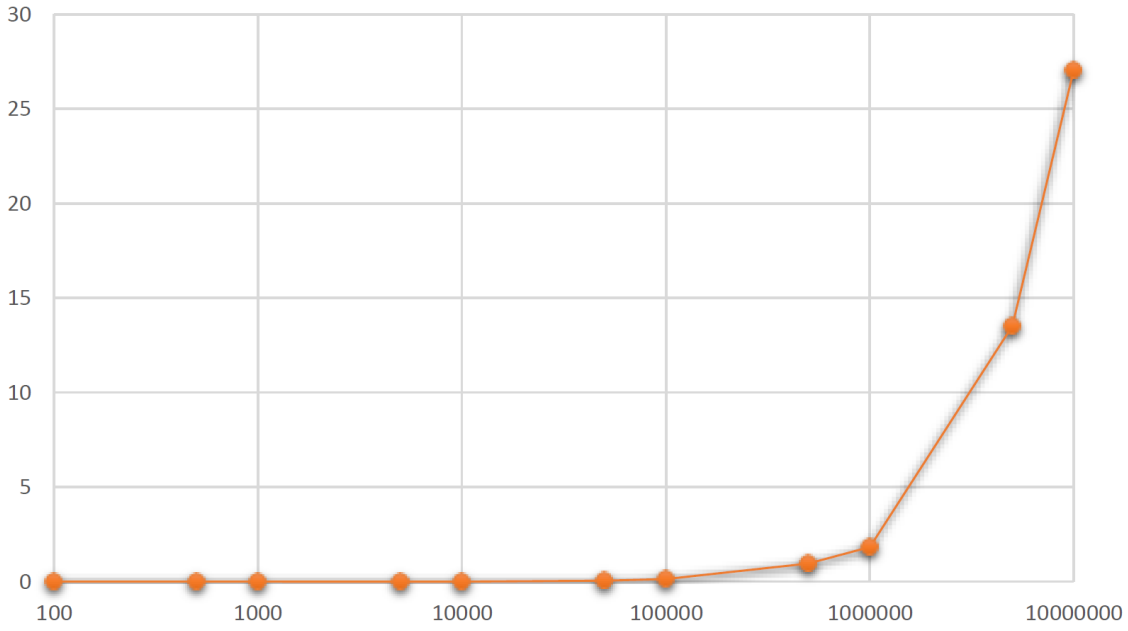
Output:

```
Enter number of elements: 7
Enter 7 elements:
38 27 43 3 9 82 10

Sorted array:
3 9 10 27 38 43 82

Time taken to sort = 0.000000 seconds
```

Graph:



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
int partition(int arr[], int low, int high)
{
   int pivot = arr[high];
   int i = low - 1;
   int j;
   for(j = low; j < high; j++)
   {
      if(arr[j] < pivot)
      {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }
   swap(&arr[i + 1], &arr[high]);
   return i + 1;
}
```

```
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main()
{
    int n, i;
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for(i = 0; i < n; i++)
    {
```

```
        printf("%d ", arr[i]);
    }
    printf("\n\nTime taken to sort = %f seconds\n", time_taken);
    return 0;
}
```

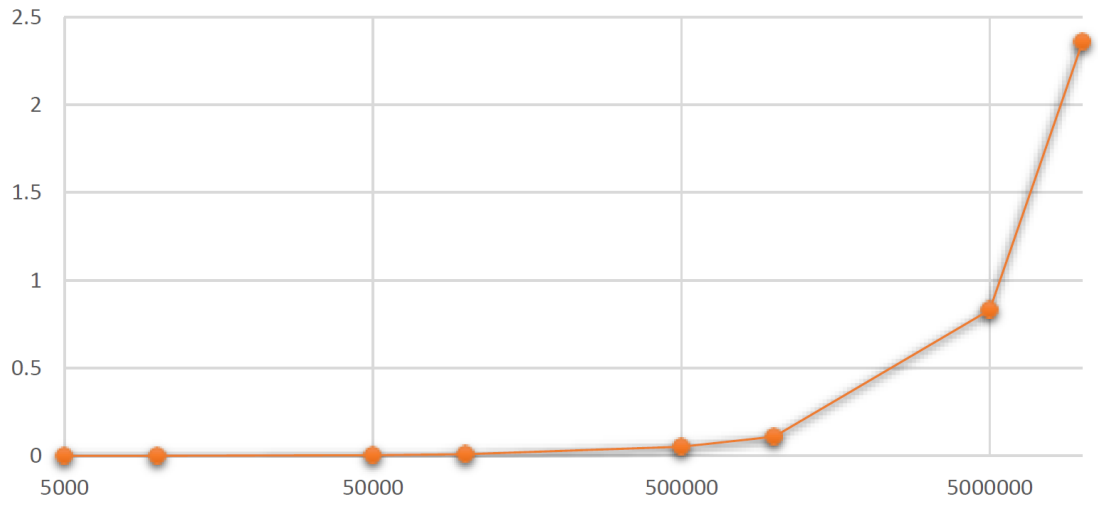
Output:

```
Enter number of elements: 7
Enter 7 elements:
38 27 43 3 9 82 10

Sorted array:
3 9 10 27 38 43 82

Time taken to sort = 0.000000 seconds
```

Graph:



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   if(left < n && arr[left] > arr[largest])
      largest = left;
   if(right < n && arr[right] > arr[largest])
      largest = right;
   if(largest != i)
   {
      swap(&arr[i], &arr[largest]);
      heapify(arr, n, largest);
   }
}
```

```
void heapSort(int arr[], int n)
{
    int i;
    for(i = n / 2 - 1; i >= 0; i--)
    {
        heapify(arr, n, i);
    }
    for(i = n - 1; i > 0; i--){
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}
int main(){
    int n, i;
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++){
        scanf("%d", &arr[i]);
    }
    start = clock();
    heapSort(arr, n);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for(i = 0; i < n; i++){
```

```
        printf("%d ", arr[i]);
    }
    printf("\n\nTime taken to sort = %f seconds\n", time_taken);
    return 0;
}
```

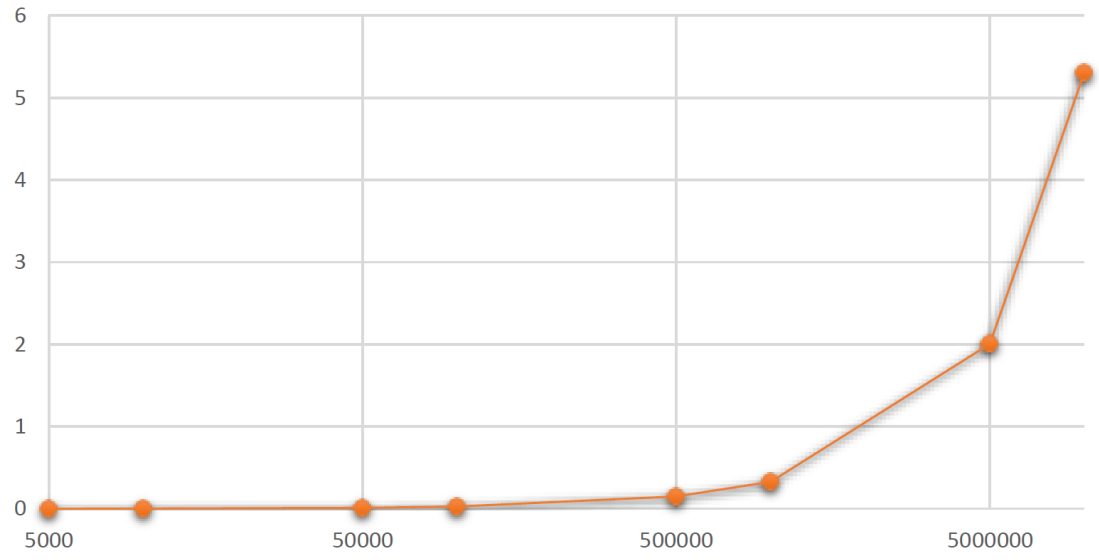
Output:

```
Enter number of elements: 7
Enter 7 elements:
9 4 3 8 10 2 5

Sorted array:
2 3 4 5 8 9 10

Time taken to sort = 0.000000 seconds
```

Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>
int max(int a, int b) {
   if (a > b)
      return a;
   else
      return b;
}
int knapsack(int W, int wt[], int val[], int n) {
   int dp[n + 1][W + 1];
   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= W; w++) {
         if (i == 0 || w == 0)
            dp[i][w] = 0;
         else if (wt[i - 1] <= w)
            dp[i][w] = max(
               val[i - 1] + dp[i - 1][w - wt[i - 1]],
               dp[i - 1][w]
            );
         else
            dp[i][w] = dp[i - 1][w];
      }
   }
   return dp[n][W];
}
```

```
int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int wt[n], val[n];
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
    printf("Enter values of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }
    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);
    int result = knapsack(W, wt, val, n);
    printf("Maximum value in knapsack = %d\n", result);
    return 0;
}
```

Output:

```
Enter number of items: 4
Enter weights of items:
5 3 4 1
Enter values of items:
7 4 5 1
Enter capacity of knapsack: 7
Maximum value in knapsack = 9
```

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include <stdio.h>
#define V 4
#define INF 99999
void printSolution(int dist[V][V]) {
   printf("\nShortest Distance Between Every Pair of Vertices:\n\n");
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         if (dist[i][j] == INF)
            printf("Vertex %d -> Vertex %d : INF\n", i, j);
         else
            printf("Vertex %d -> Vertex %d : %d\n",
                  i, j, dist[i][j]);
      }
   }
}
void floydWarshall(int graph[V][V]) {
   int dist[V][V];
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         dist[i][j] = graph[i][j];
      }
   }
   for (int k = 0; k < V; k++) {
      for (int i = 0; i < V; i++) {
         for (int j = 0; j < V; j++) {
```

```
                if (dist[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    printSolution(dist);
}
int main() {
    int graph[V][V] = {
        {0,   3,   INF, 7},
        {8,   0,   2,   INF},
        {5,   INF, 0,   1},
        {2,   INF, INF, 0}
    };
    floydWarshall(graph);
    return 0;
}
```

Output:

```
Shortest Distance Between Every Pair of Vertices:

Vertex 0 -> Vertex 0 : 0
Vertex 0 -> Vertex 1 : 3
Vertex 0 -> Vertex 2 : 5
Vertex 0 -> Vertex 3 : 6
Vertex 1 -> Vertex 0 : 5
Vertex 1 -> Vertex 1 : 0
Vertex 1 -> Vertex 2 : 2
Vertex 1 -> Vertex 3 : 3
Vertex 2 -> Vertex 0 : 3
Vertex 2 -> Vertex 1 : 6
Vertex 2 -> Vertex 2 : 0
Vertex 2 -> Vertex 3 : 1
Vertex 3 -> Vertex 0 : 2
Vertex 3 -> Vertex 1 : 5
Vertex 3 -> Vertex 2 : 7
Vertex 3 -> Vertex 3 : 0
```

Lab program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Code:

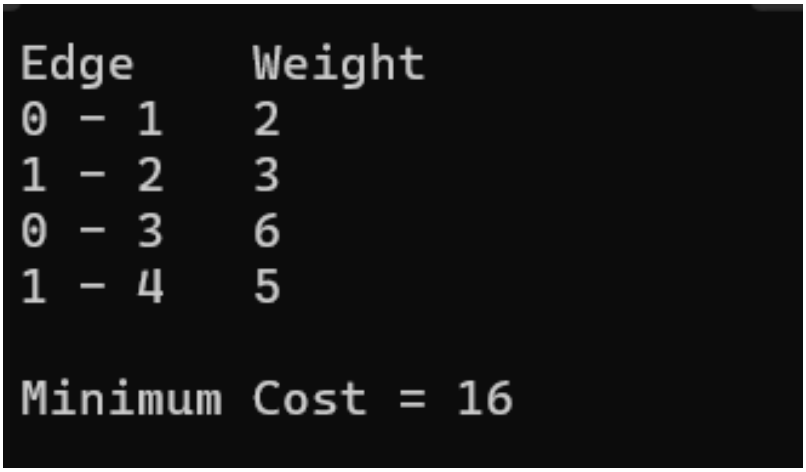
```
#include <stdio.h>
#include <limits.h>
#define V 5
int minKey(int key[], int mstSet[]) {
   int min = INT_MAX, min_index;
   for (int v = 0; v < V; v++) {
      if (mstSet[v] == 0 && key[v] < min) {
         min = key[v];
         min_index = v;
      }
   }
   return min_index;
}
void printMST(int parent[], int graph[V][V]) {
   int totalCost = 0;
   printf("Edge \tWeight\n");
   for (int i = 1; i < V; i++) {
      printf("%d - %d \t%d\n",
            parent[i], i,
            graph[i][parent[i]]);

      totalCost += graph[i][parent[i]];
   }
   printf("\nMinimum Cost = %d\n", totalCost);
}
```

```
void primMST(int graph[V][V]) {
    int parent[V];
    int key[V];
    int mstSet[V];
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++) {
            if (graph[u][v] &&
                mstSet[v] == 0 &&
                graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printMST(parent, graph);
}
int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
```

```
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    primMST(graph);
    return 0;
}
```

Output:



Lab program 8b:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

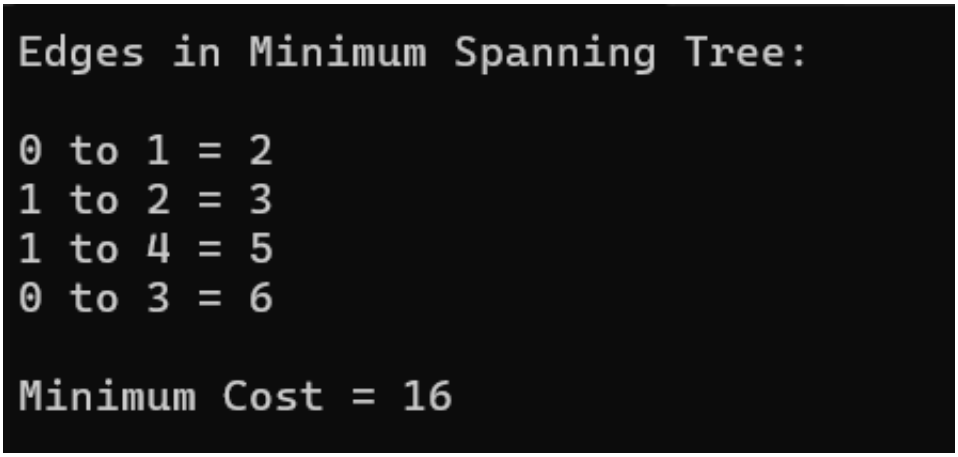
Code:

```
#include <stdio.h>
#define INF 9999
int p[10] = {0};
int applyfind(int i)
{
   while (p[i] != 0)
      i = p[i];
   return i;
}
int applyunion(int i, int j)
{
   if (i != j)
   {
      p[j] = i;
      return 1;
   }
   return 0;
}
int main()
{
   int cost[10][10] = {
      {INF, 2, INF, 6, INF},
      {2, INF, 3, 8, 5},
      {INF, 3, INF, INF, 7},
      {6, 8, INF, INF, 9},
```

```
    {INF, 5, 7, 9, INF}
};
int n = 5;
int i, j;
int ne = 1;
int mincost = 0;
int min, a, b, u, v;
printf("Edges in Minimum Spanning Tree:\n\n");
while (ne < n)
{
    min = INF;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (cost[i][j] < min)
            {
                min = cost[i][j];
                a = u = i;
                b = v = j;
            }
        }
    }
    u = applyfind(u);
    v = applyfind(v);
    if (applyunion(u, v))
    {
        printf("%d to %d = %d\n", a, b, min);
        mincost += min;
```

```
            ne++;
        }
        cost[a][b] = cost[b][a] = INF;
    }
    printf("\nMinimum Cost = %d\n", mincost);
    return 0;
}
```

Output:



Lab program 9:

Implement Fractional Knapsack using Greedy technique.

Code:

```
#include <stdio.h>
struct Item {
   int value;
   int weight;
};
void sortItems(struct Item arr[], int n) {
   int i, j;
   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         double r1 = (double)arr[j].value / arr[j].weight;
         double r2 = (double)arr[j + 1].value / arr[j + 1].weight;
         if (r1 < r2) {
            int tempval = arr[j].value;
            arr[j].value = arr[j+1].value;
            arr[j+1].value = tempval;
            int tempwt = arr[j].weight;
            arr[j].weight = arr[j+1].weight;
            arr[j+1].weight = tempwt;
         }
      }
   }
}
double fractionalKnapsack(struct Item arr[], int n, int capacity) {
   sortItems(arr, n);
   double totalValue = 0.0;
   for (int i = 0; i < n; i++) {
```

```
        if (capacity >= arr[i].weight) {
            capacity -= arr[i].weight;
            totalValue += arr[i].value;
        } else {
            totalValue += arr[i].value * ((double)capacity / arr[i].weight);
            break;
        }
    }
    return totalValue;
}
int main() {
    int n, capacity, i;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item arr[n];
    printf("Enter value and weight of each item:\n");
    for (i = 0; i < n; i++) {
        scanf("%d %d", &arr[i].value, &arr[i].weight);
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    double maxValue = fractionalKnapsack(arr, n, capacity);
    printf("Maximum value in Knapsack = %.2f\n", maxValue);
    return 0;
}
```

Output:

```
Enter number of items: 3
Enter value and weight of each item:
60 10
100 20
120 30
Enter knapsack capacity: 50
Maximum value in Knapsack = 240.00
```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include <stdio.h>
#include <limits.h>
#define MAX_VERTICES 100
int minDistance(int dist[], int sptSet[], int vertices) {
   int min = INT_MAX, minIndex;
   for (int v = 0; v < vertices; v++) {
      if (!sptSet[v] && dist[v] < min) {
         min = dist[v];
         minIndex = v;
      }
   }
   return minIndex;
}
void printSolution(int dist[], int vertices) {
   printf("Vertex \tDistance from Source\n");
   for (int i = 0; i < vertices; i++) {
      printf("%d \t%d\n", i, dist[i]);
   }
}
void dijkstra(int graph[MAX_VERTICES][MAX_VERTICES], int src, int vertices) {
   int dist[MAX_VERTICES];
   int sptSet[MAX_VERTICES];
   for (int i = 0; i < vertices; i++) {
      dist[i] = INT_MAX;
      sptSet[i] = 0;
```

```
    }
    dist[src] = 0;
    for (int count = 0; count < vertices - 1; count++) {
        int u = minDistance(dist, sptSet, vertices);
        sptSet[u] = 1;
        for (int v = 0; v < vertices; v++) {
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] +
graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printSolution(dist, vertices);
}
int main() {
    int vertices;
    printf("Input the number of vertices: ");
    scanf("%d", &vertices);
    if (vertices <= 0 || vertices > MAX_VERTICES) {
        printf("Invalid number of vertices. Exiting...\n");
        return 1;
    }
    int graph[MAX_VERTICES][MAX_VERTICES];
    printf("Input the adjacency matrix for the graph (use INT_MAX for infinity):\n");
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    int source;
```

```
    printf("Input the source vertex: ");
    scanf("%d", &source);
    if (source < 0 || source >= vertices) {
        printf("Invalid source vertex. Exiting...\n");
        return 1;
    }
    dijkstra(graph, source, vertices);
    return 0;
}
```

Output:

```
Input the number of vertices: 5
Input the adjacency matrix for the graph (use INT_MAX for infinity):
0 3 2 0 0
3 0 0 1 0
2 0 0 1 4
0 1 1 0 2
0 0 4 2 0
Input the source vertex: 0
Vertex  Distance from Source
0       0
1       3
2       2
3       3
4       5
```

Lab program 11:

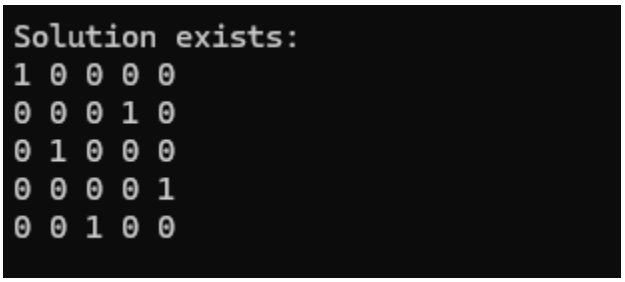
Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>
#define N 5
int board[N][N];
void printBoard() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}
int isSafe(int row, int col) {
    for (int i = 0; i < col; i++) {
        if (board[row][i])
            return 0;
    }
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j])
            return 0;
    }
    for (int i = row, j = col; i < N && j >= 0; i++, j--) {
        if (board[i][j])
            return 0;
    }
    return 1;
```

```
}
int solveNQ(int col) {
    if (col >= N)
        return 1;
    for (int i = 0; i < N; i++) {
        if (isSafe(i, col)) {
            board[i][col] = 1;
            if (solveNQ(col + 1))
                return 1;
            board[i][col] = 0;
        }
    }
    return 0;
}
int main() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }
    if (solveNQ(0)) {
        printf("Solution exists:\n");
        printBoard();
    } else {
        printf("No solution exists.\n");
    }
    return 0;
}
```

Output:



LeetCode Problem 1:

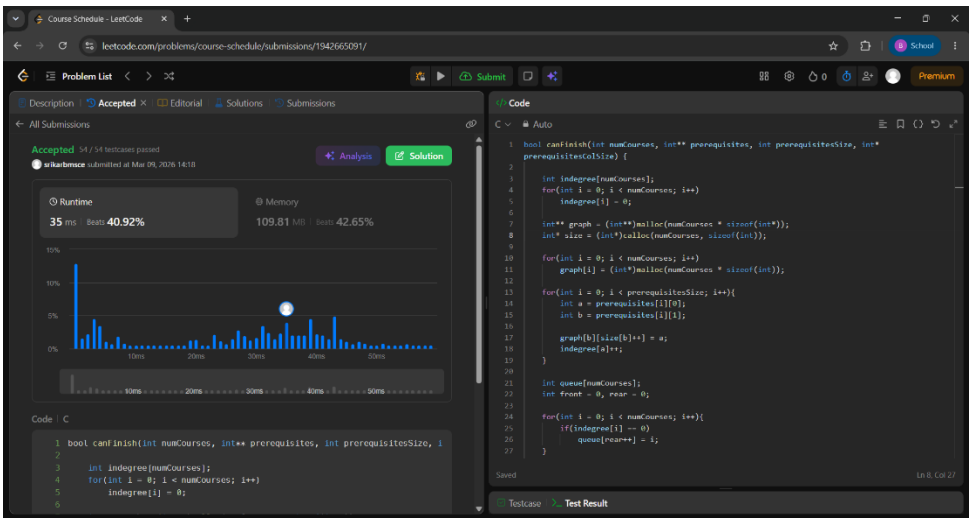
207. Course Schedule

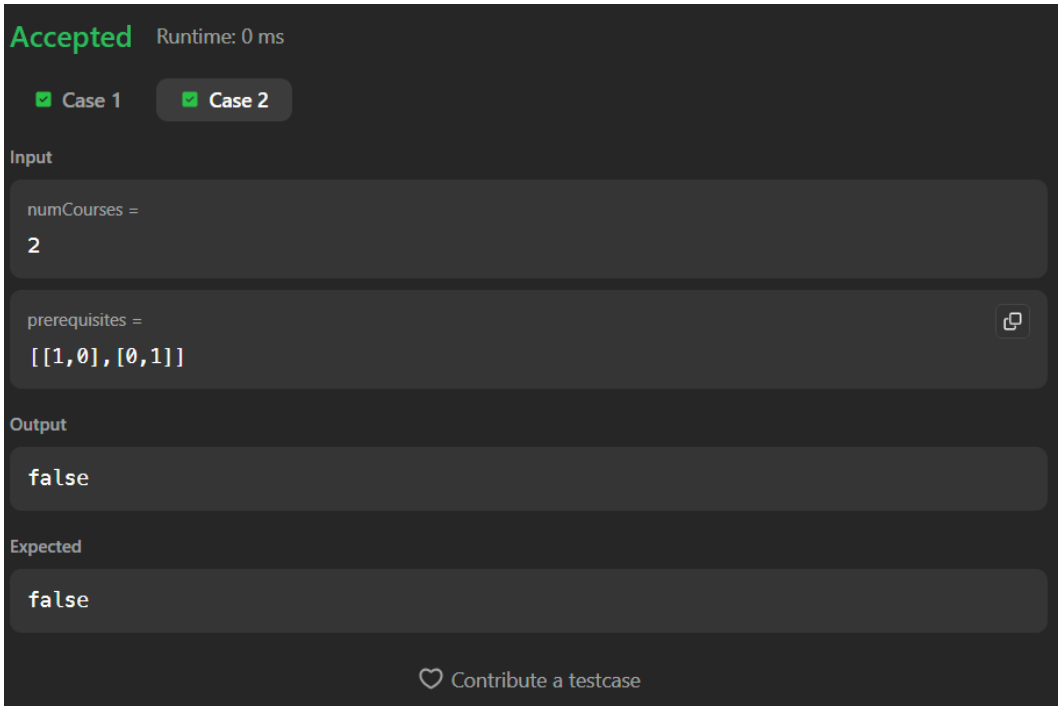
Code:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int indegree[numCourses];
    for(int i = 0; i < numCourses; i++)
        indegree[i] = 0;
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* size = (int*)calloc(numCourses, sizeof(int));
    for(int i = 0; i < numCourses; i++)
        graph[i] = (int*)malloc(numCourses * sizeof(int));
    for(int i = 0; i < prerequisitesSize; i++){
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][size[b]++] = a;
        indegree[a]++;
    }
    int queue[numCourses];
    int front = 0, rear = 0;
    for(int i = 0; i < numCourses; i++){
        if(indegree[i] == 0)
            queue[rear++] = i;
    }
    int count = 0;
    while(front < rear){
        int course = queue[front++];
        count++;
```

```
        for(int i = 0; i < size[course]; i++){
            int next = graph[course][i];
            indegree[next]--;
            if(indegree[next] == 0)
                queue[rear++] = next;
        }
    }
    return count == numCourses;
}
```

Output:




Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0],[0,1]]
Output
false
Expected
false
Contribute a testcase

LeetCode Problem 2:

46. Permutations

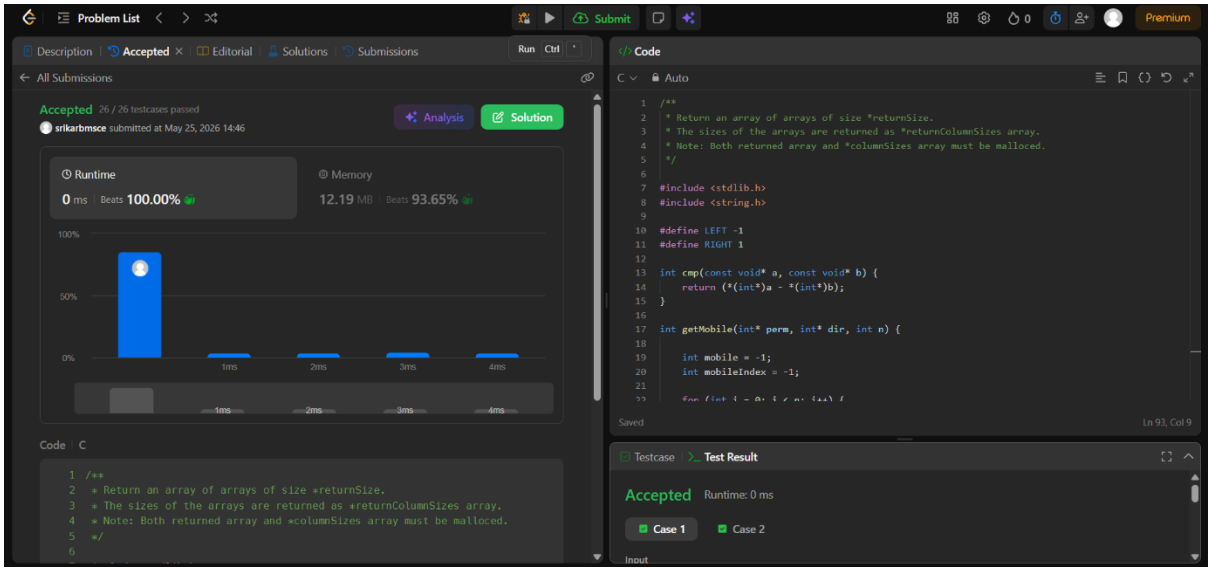
Code:

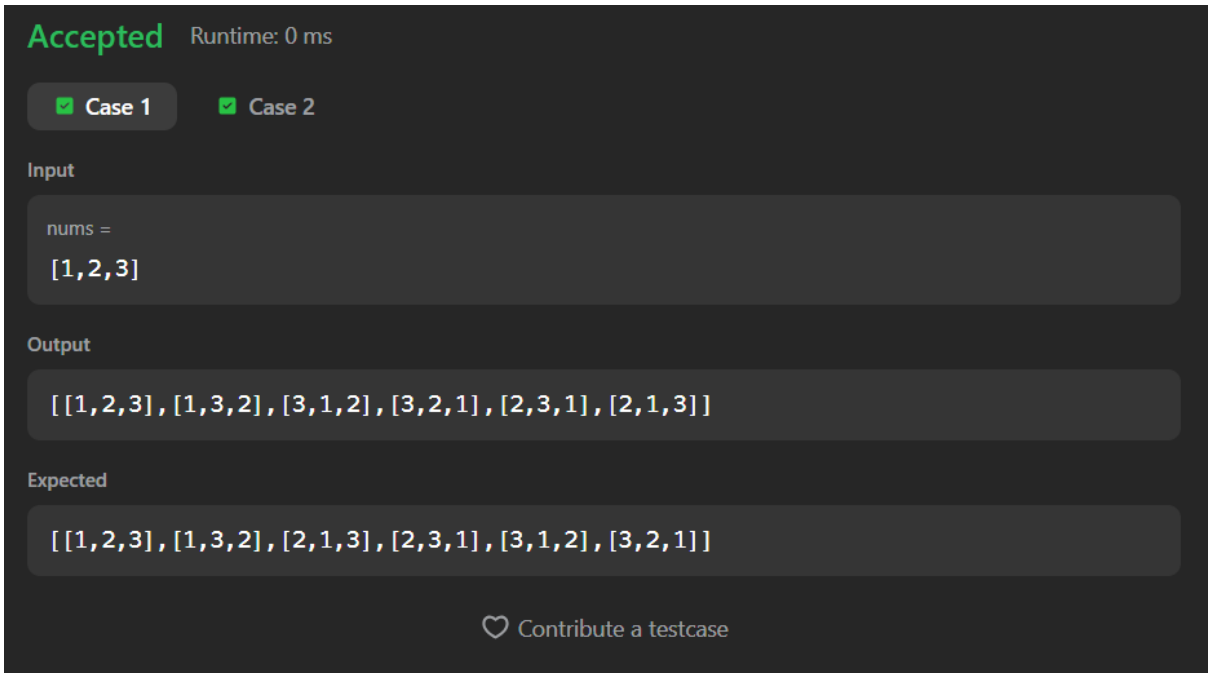
```
#include <stdlib.h>
#include <string.h>
#define LEFT -1
#define RIGHT 1
int cmp(const void* a, const void* b) {
   return (*(int*)a - *(int*)b);
}
int getMobile(int* perm, int* dir, int n) {
   int mobile = -1;
   int mobileIndex = -1;
   for (int i = 0; i < n; i++) {
      if (dir[i] == LEFT && i > 0 && perm[i] > perm[i - 1]) {
         if (perm[i] > mobile) {
            mobile = perm[i];
            mobileIndex = i;
         }
      }
      if (dir[i] == RIGHT && i < n - 1 && perm[i] > perm[i + 1]) {
         if (perm[i] > mobile) {
            mobile = perm[i];
            mobileIndex = i;
         }
      }
   }
   return mobileIndex;
```

```
}
int** permute(int* nums, int numsSize, int* returnSize, int** returnColumnSizes) {
    int total = 1;
    for (int i = 1; i <= numsSize; i++)
        total *= i;
    int** result = (int**)malloc(total * sizeof(int*));
    *returnColumnSizes = (int*)malloc(total * sizeof(int));
    int* sorted = (int*)malloc(numsSize * sizeof(int));
    memcpy(sorted, nums, numsSize * sizeof(int));
    qsort(sorted, numsSize, sizeof(int), cmp);
    int* perm = (int*)malloc(numsSize * sizeof(int));
    for (int i = 0; i < numsSize; i++)
        perm[i] = i + 1;
    int* dir = (int*)malloc(numsSize * sizeof(int));
    for (int i = 0; i < numsSize; i++)
        dir[i] = LEFT;
    *returnSize = 0;
    while (1) {
        result[*returnSize] = (int*)malloc(numsSize * sizeof(int));
        for (int i = 0; i < numsSize; i++) {
            result[*returnSize][i] = sorted[perm[i] - 1];
        }
        (*returnColumnSizes)[*returnSize] = numsSize;
        (*returnSize)++;
        int mobileIndex = getMobile(perm, dir, numsSize);
        if (mobileIndex == -1)
            break;
        int swapIndex = mobileIndex + dir[mobileIndex];
        int temp = perm[mobileIndex];
```

```
            perm[mobileIndex] = perm[swapIndex];
            perm[swapIndex] = temp;
            temp = dir[mobileIndex];
            dir[mobileIndex] = dir[swapIndex];
            dir[swapIndex] = temp;
            mobileIndex = swapIndex;
            int mobileValue = perm[mobileIndex];
            for (int i = 0; i < numsSize; i++) {
                if (perm[i] > mobileValue)
                    dir[i] *= -1;
            }
        }
        free(sorted);
        free(perm);
        free(dir);
        return result;
    }
```

Output:




Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[1,2,3]
Output
[[1,2,3],[1,3,2],[3,1,2],[3,2,1],[2,3,1],[2,1,3]]
Expected
[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
Contribute a testcase


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[0,1]
Output
[[0,1],[1,0]]
Expected
[[0,1],[1,0]]
Contribute a testcase

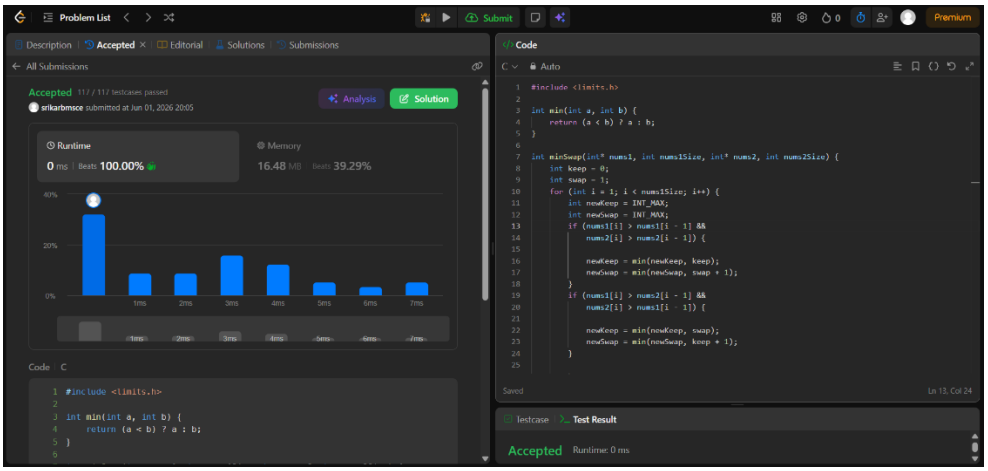
LeetCode Problem 3:

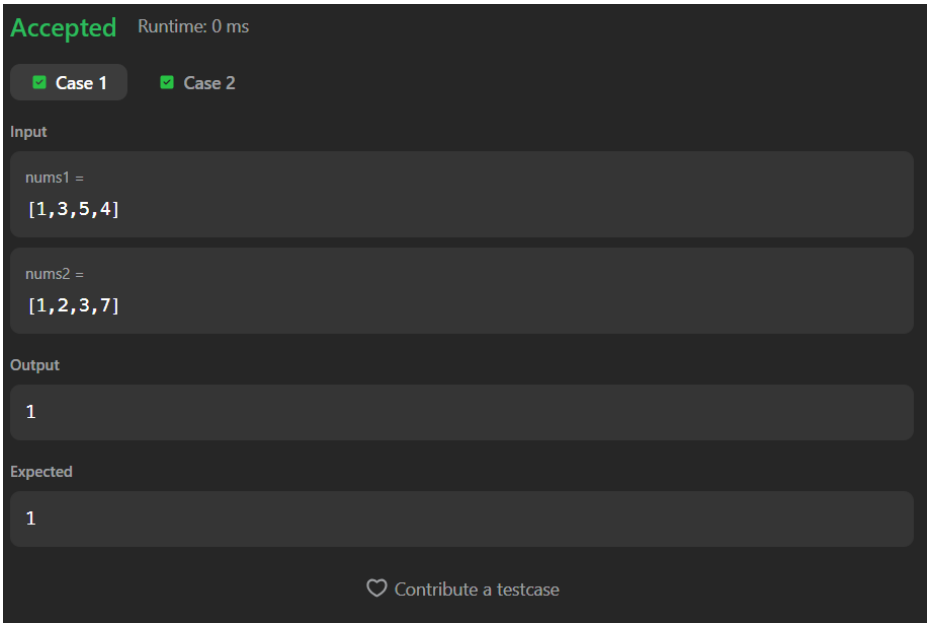
801. Minimum Swaps to Make Sequences Increasing

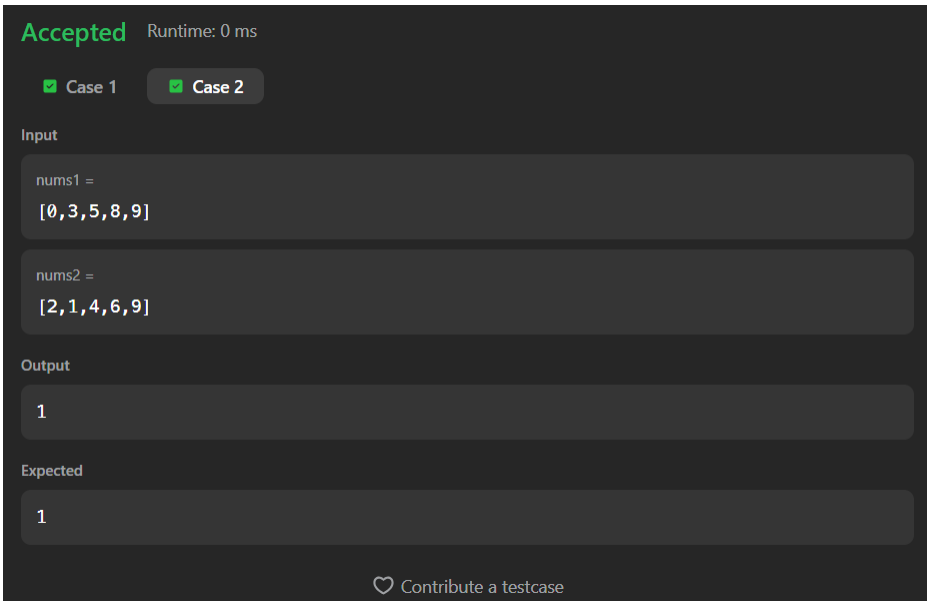
Code:

```
#include <limits.h>
int min(int a, int b) {
    return (a < b) ? a : b;
}
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;
    for (int i = 1; i < nums1Size; i++) {
        int newKeep = INT_MAX;
        int newSwap = INT_MAX;
        if (nums1[i] > nums1[i - 1] &&
            nums2[i] > nums2[i - 1]) {
            newKeep = min(newKeep, keep);
            newSwap = min(newSwap, swap + 1);
        }
        if (nums1[i] > nums2[i - 1] &&
            nums2[i] > nums1[i - 1]) {
            newKeep = min(newKeep, swap);
            newSwap = min(newSwap, keep + 1);
        }
        keep = newKeep;
        swap = newSwap;
    }
    return min(keep, swap);
}
```

Output:


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 117 / 117 testcases passed
srikarbmsce submitted at Jun 01, 2026 20:05
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
16.48 MB | Beats 39.29%
Code
C
Auto
Testcase
Test Result
Accepted Runtime: 0 ms


Accepted Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[1,3,5,4]
nums2 =
[1,2,3,7]
Output
1
Expected
1
Contribute a testcase


Accepted Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[0,3,5,8,9]
nums2 =
[2,1,4,6,9]
Output
1
Expected
1
Contribute a testcase

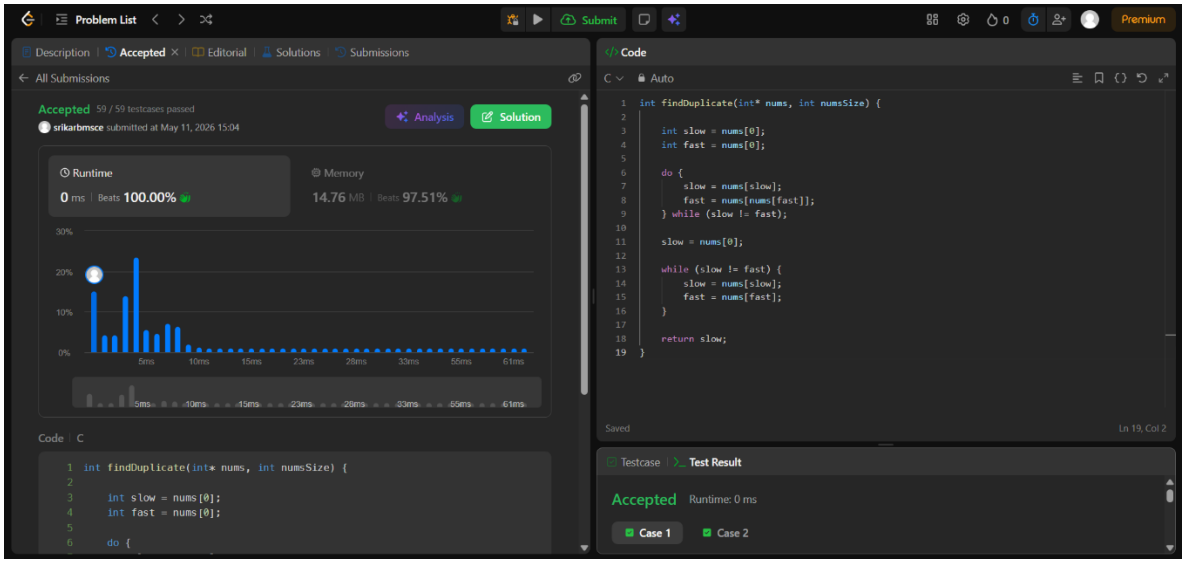
LeetCode Problem 4:

287. Find the Duplicate Number

Code:

```
int findDuplicate(int* nums, int numsSize) {
    int slow = nums[0];
    int fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);
    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

Output:




Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[1,3,4,2,2]
Output
2
Expected
2
Contribute a testcase


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[3,1,3,4,2]
Output
3
Expected
3
Contribute a testcase

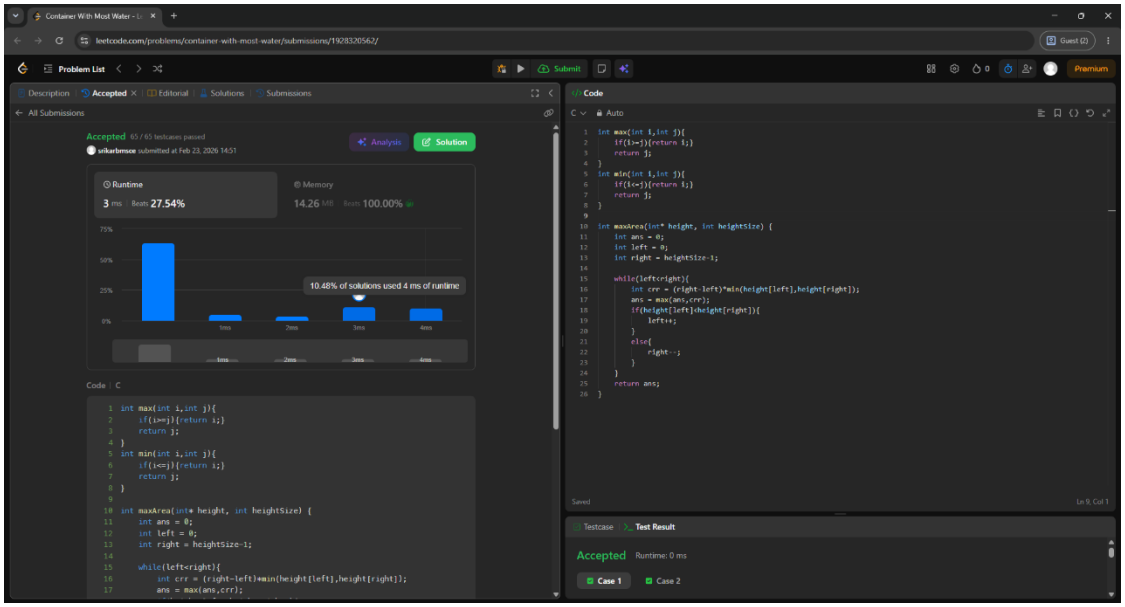
LeetCode Problem 5:

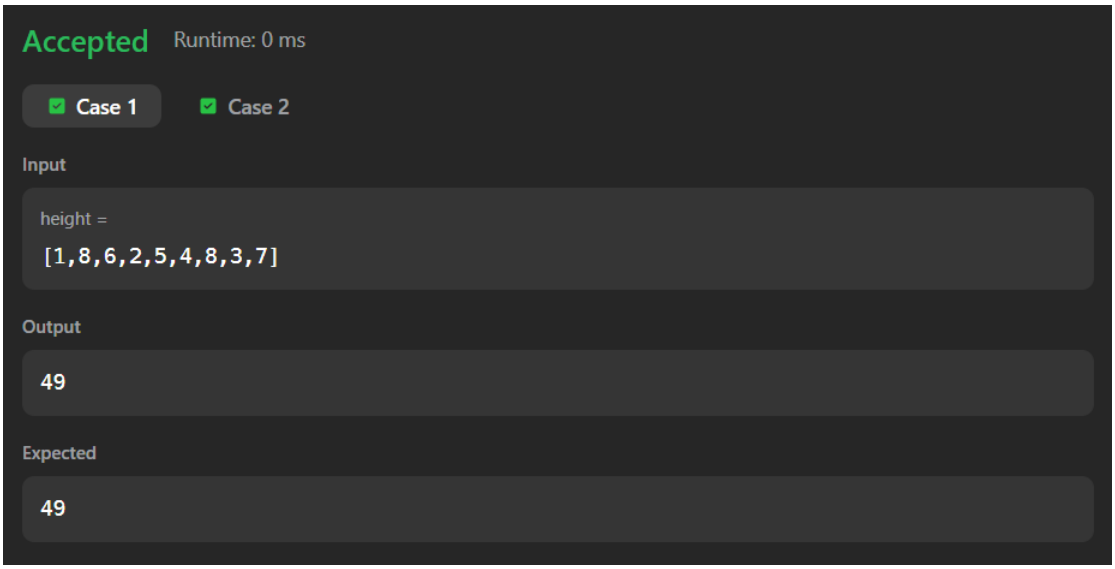
11. Container With Most Water

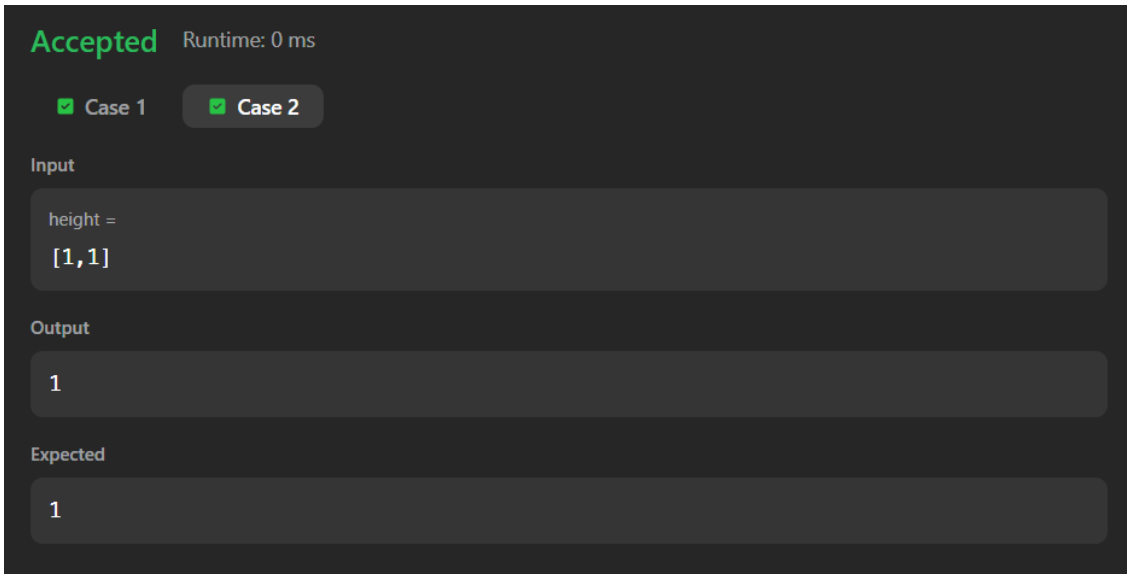
Code:

```
int max(int i,int j){
    if(i>=j){return i;}
    return j;
}
int min(int i,int j){
    if(i<=j){return i;}
    return j;
}
int maxArea(int* height, int heightSize) {
    int ans = 0;
    int left = 0;
    int right = heightSize-1;
    while(left<right){
        int crr = (right-left)*min(height[left],height[right]);
        ans = max(ans,crr);
        if(height[left]<height[right]){
            left++;
        }
        else{
            right--;
        }
    }
    return ans;
}
```

Output:


Container With Most Water
leetcode.com/problems/container-with-most-water/submissions/1928320562/
Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 65 / 65 testcases passed
srikarbmsce submitted at Feb 23, 2026 14:51
Analysis
Solution
Runtime
3 ms Beats 27.54%
Memory
14.26 MB Beats 100.00%
10.48% of solutions used 4 ms of runtime
Code | C
Code
C Auto
Saved
Ln 9, Col 1
Testcase
Test Result
Accepted Runtime: 0 ms
Case 1
Case 2


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
height =
[1,8,6,2,5,4,8,3,7]
Output
49
Expected
49


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
height =
[1,1]
Output
1
Expected
1

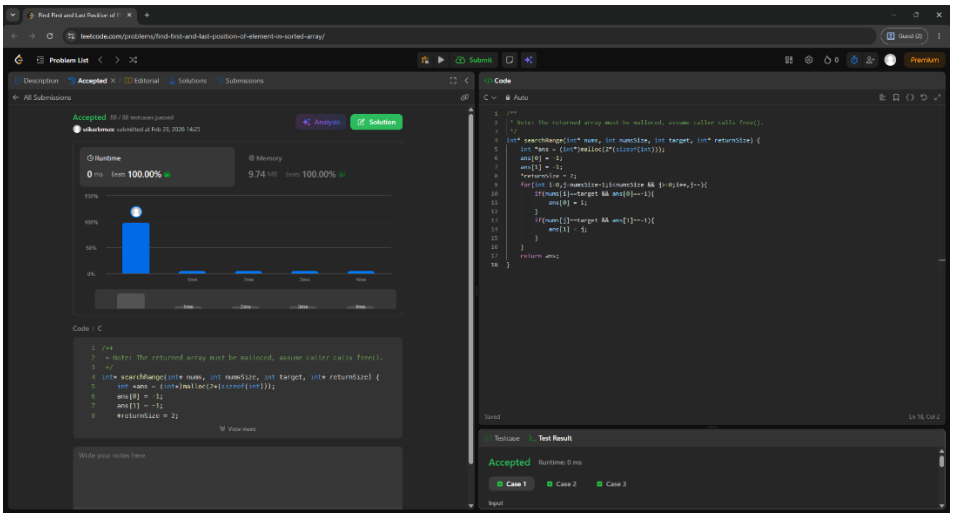
LeetCode Problem 6:

34. Find First and Last Position of Element in Sorted Array

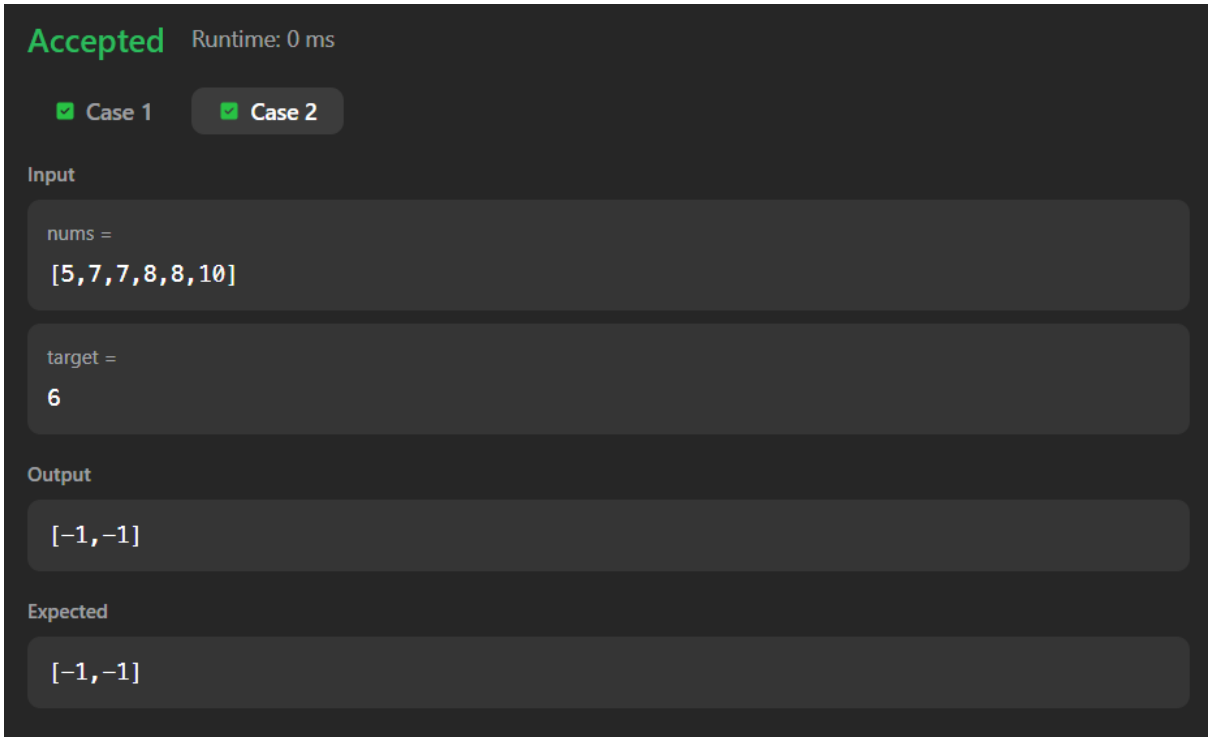
Code:

```
int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
    int *ans = (int*)malloc(2*(sizeof(int)));
    ans[0] = -1;
    ans[1] = -1;
    *returnSize = 2;
    for(int i=0,j=numsSize-1;i<numsSize && j>=0;i++,j--){
        if(nums[i]==target && ans[0]==-1){
            ans[0] = i;
        }
        if(nums[j]==target && ans[1]==-1){
            ans[1] = j;
        }
    }
    return ans;
}
```

Output:




Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[5,7,7,8,8,10]
target =
8
Output
[3,4]
Expected
[3,4]


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[5,7,7,8,8,10]
target =
6
Output
[-1,-1]
Expected
[-1,-1]

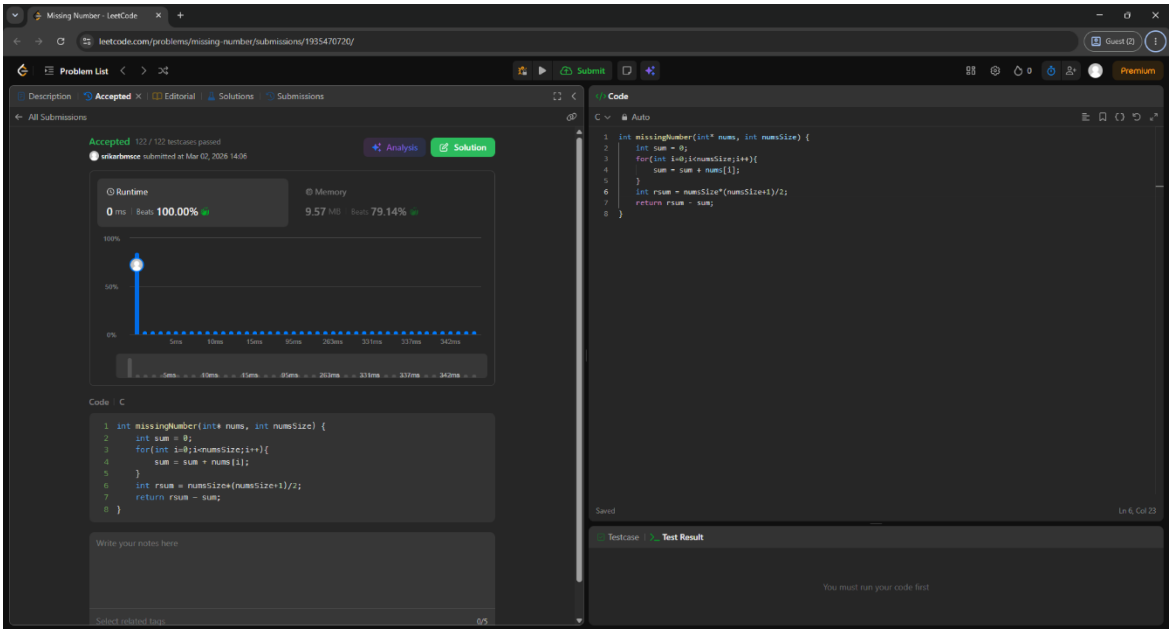
LeetCode Problem 7:

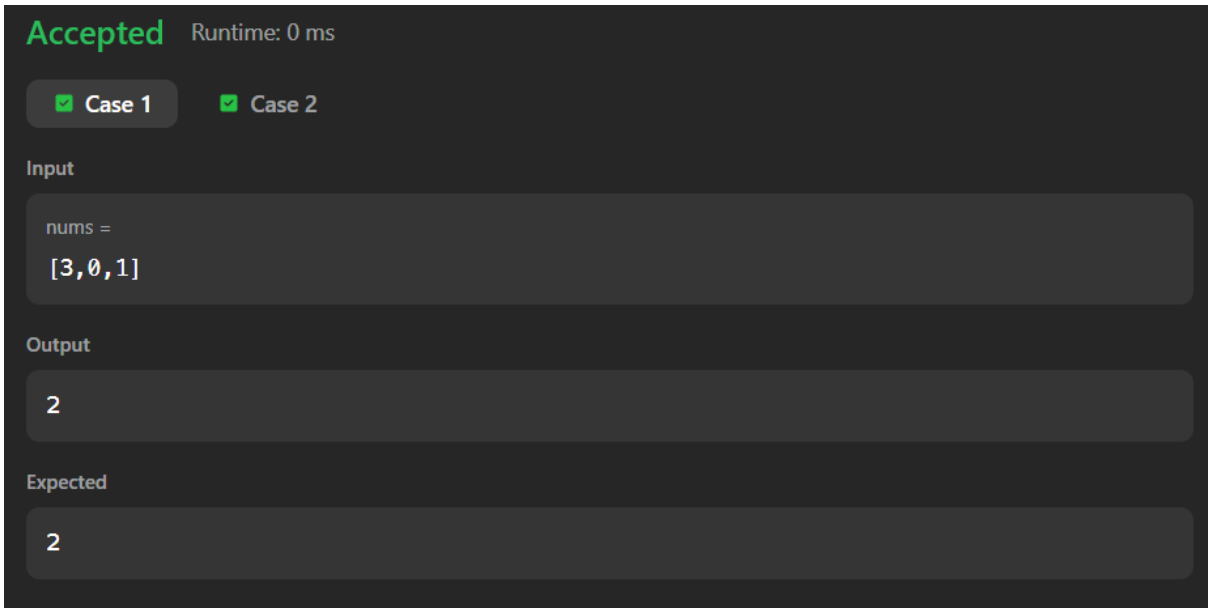
268. Missing Number

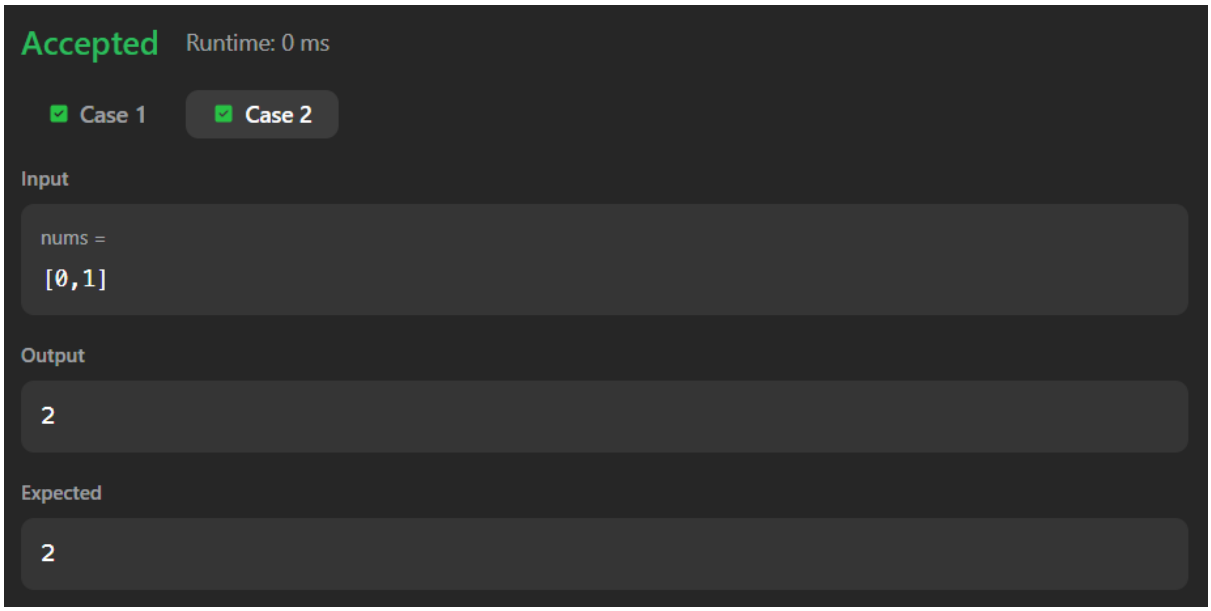
Code:

```
int missingNumber(int* nums, int numsSize) {
    int sum = 0;
    for(int i=0;i<numsSize;i++){
        sum = sum + nums[i];
    }
    int rsum = numsSize*(numsSize+1)/2;
    return rsum - sum;
}
```

Output:




Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[3,0,1]
Output
2
Expected
2


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[0,1]
Output
2
Expected
2

LeetCode Problem 8:

1636. Sort Array by Increasing Frequency

Code:

```
int freq(int k,int* num,int numsSize){
    int c = 0;
    for(int i=0;i<numsSize;i++){
        if(num[i]==k){c = c+1;}
    }
    return c;
}
int* frequencySort(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    int *fa = (int*)malloc(numsSize*sizeof(int));
    for(int i=0;i<numsSize;i++){
        fa[i] = freq(nums[i],nums,numsSize);
    }
    for(int j=0;j<numsSize-1;j++){
        for(int k=0;k<numsSize-1;k++){
            if((fa[k]>fa[k+1]) || (fa[k]==fa[k+1] && nums[k]<nums[k+1])){
                int temp = nums[k];
                nums[k] = nums[k+1];
                nums[k+1] = temp;
                int tempf = fa[k];
                fa[k] = fa[k+1];
                fa[k+1] = tempf;
            }
        }
    }
```

```
    return nums;
}
```

Output: